# 计算机组成原理实验报告

## 基本信息

- 实验名称：Lab1-2
- 姓名：陈一璟
- 学号：24300120183

## 一、实验目的

本实验旨在通过设计一个有限状态机（FSM）来检测特定序列"1011"，掌握数字电路中状态机的设计与实现方法。具体任务包括：

1. 绘制完整状态转移图，表示1011检测模块的状态转移；
2. 设计自动机检测1011序列，补全代码；
3. 运行Vivado仿真，分析波形，验证状态机是否能够正确检测包含重叠情况的1011序列。

## 二、实验原理

1. 自动机原理

- 自动机是一种用于处理序列的抽象模型，能够根据输入序列进行状态转换并输出对应结果。
- 状态：对历史输入序列的抽象，用于决定下一状态及输出
- 输入：状态机的输入，用于触发状态转换。
- 输出：状态机的输出，用于表示当前状态的结果。
- 状态转换：根据当前状态和输入，将状态转换为下一个状态。

2. 有限状态自动机类型

- Mealy型：输出不仅取决于当前状态，还取决于当前输入
- Moore型：输出仅取决于当前状态
- 本实验采用Moore型状态机，输出由当前状态决定。
- 特别地，输出逻辑采用组合译码方式实现，可用连续赋值语句描述，使状态更新后输出立即响应，而不会再增加额外寄存器延迟。

3. 状态转移设计

- 初始状态：IDLE（空闲状态）
- 目标序列：1011
- 状态定义：IDLE（初始状态）、S1（检测到1）、S2（检测到10）、S3（检测到101）、S4（检测到1011）
- 跳转逻辑：以当前输入序列的最长后缀与目标序列前缀匹配为准，从而支持重叠序列的检测（如1011011）。

4. 状态编码方式

- 二进制编码：使用最少的位数表示状态
- 格雷码：相邻状态只有一位不同，减少状态转换时的毛刺
- One-hot编码：每个状态用一位表示，虽然占用更多位宽，但状态转移逻辑简单，易于实现

- 本实验采用One-hot编码，可减少状态译码的组合逻辑复杂度，在 FPGA 实现中通常有利于提高时序性能，并简化设计。

# 三、实验步骤

## 1. 关键代码

1. Onehot 编码补全

```
parameter IDLE = 5'b00001,    // 空闲状态
          S1   = 5'b00010,    // 检测到1
          S2   = 5'b00100,    // 检测到10
          S3   = 5'b01000,    // 检测到101
          S4   = 5'b10000;    // 检测到1011（目标序列）
```

2. 状态转移逻辑

状态转移采用同步时序逻辑实现，复位为低有效异步复位。

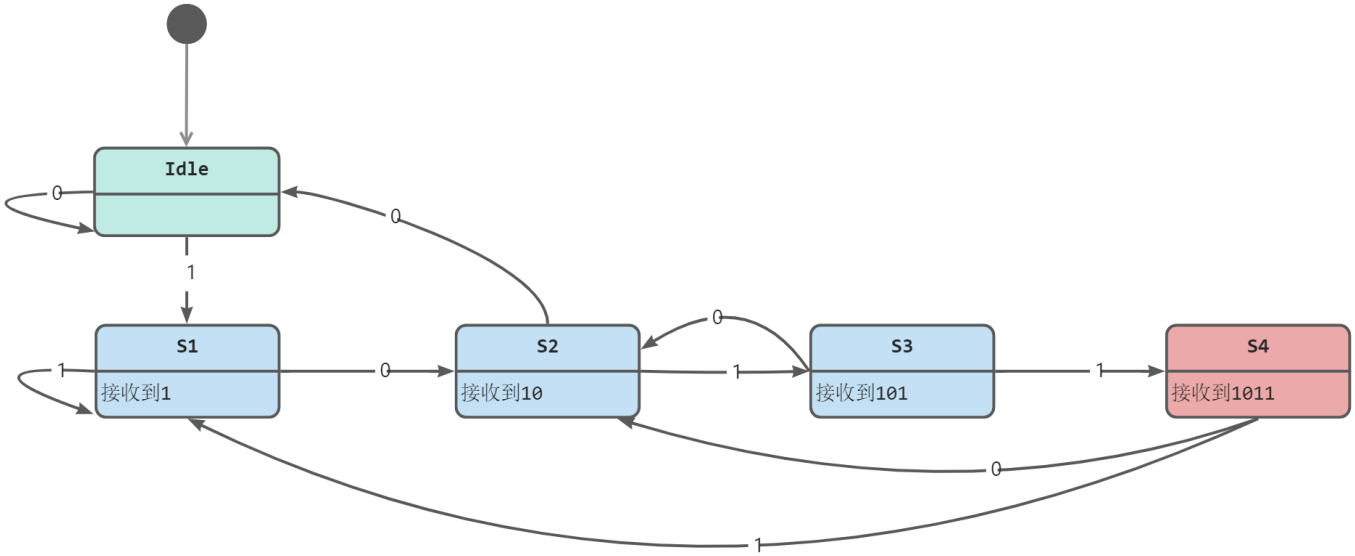
```
always@(posedge Clk_50 or negedge Rst_n)     // 在上升沿或复位时更新状态
begin
    if (!Rst_n) begin
        state <= IDLE;  // 复位时，状态为空闲状态
    end else begin
        case(state)     // 根据当前状态和输入数据更新状态
            IDLE: if (data_in)    // IDLE状态下输入为1时，状态转换为S1
                    state <= S1;
                else
                    state <= IDLE;
            S1: if (!data_in)    // S1状态下输入为0时，状态转换为S2
                    state <= S2;
                else
                    state <= S1;
            S2: if (data_in)     // S2状态下输入为1时，状态转换为S3
                    state <= S3;
                else
                    state <= IDLE;
            S3: if (data_in)     // S3状态下输入为1时，状态转换为S4，此时探测到1011序
列
                    state <= S4;
                else
                    state <= S2;
            S4: if (data_in)
                    state <= S1;
                else
                    state <= S2;
            default: state <= IDLE;
        endcase
    end
end
```

3. 输出逻辑

```
// 当且仅当检测到目标序列1011时输出高电平 ，其余时刻输出低电平
// 使用连续赋值的组合逻辑实现输出，使其无需等待一个时钟周期延迟即可响应
assign data_out = (state == S4);
```

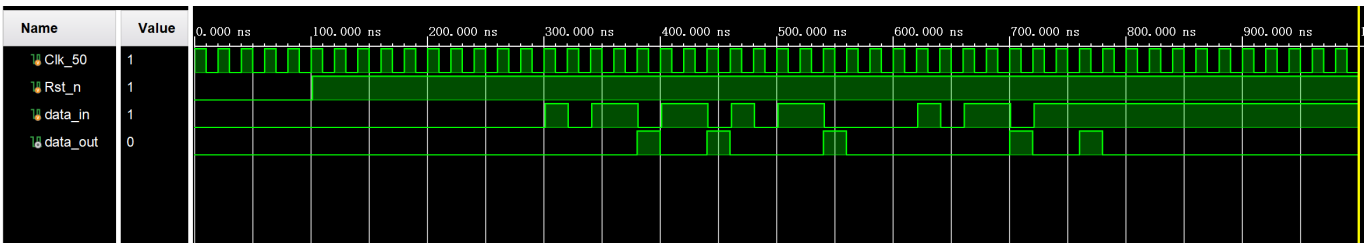
# 四、实验结果

## 1. 完整状态转移图




- 其中，`IDLE`状态为初始状态（未匹配到任何可接受序列），蓝色块为中间状态（匹配到部分可接受序列），红色块为接受状态（匹配到完整`1011`序列）。

## 2. 波形结果（适用于Vivado仿真）




- `Clk_50`为50MHz时钟信号，`Rst_n`为复位信号，`data_in`为输入数据，`data_out`为输出数据。
- 输入为连续序列：1011 0110 1011 0000 1011011，状态机能够在不复位情况下持续检测目标序列。
- 仿真结果表明，该状态机能够正确检测非重叠与重叠的 1011 序列，并在检测完成时输出单周期高电平。

**1. 初始化阶段：**

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 0ns-101ns | 0 | 0 | IDLE | 0 | 复位状态 |
| 101ns-301ns | 1 | 0 | IDLE | 0 | 释放复位，等待稳定 |

**2. 第一组测试序列（1011）：**

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 301ns-321ns | 1 | 1 | S1 | 0 | 检测到第一个1 |
| 321ns-341ns | 1 | 0 | S2 | 0 | 检测到10 |
| 341ns-361ns | 1 | 1 | S3 | 0 | 检测到101 |
| 361ns-381ns | 1 | 1 | S4 | 1 | 检测到1011，输出高电平 |

**3. 第二组测试序列（0110）：**

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 381ns-401ns | 1 | 0 | S2 | 0 | 从S4状态输入0，转移到S2 |
| 401ns-421ns | 1 | 1 | S3 | 0 | 检测到101 |
| 421ns-441ns | 1 | 1 | S4 | 1 | 检测到1011，输出高电平 |
| 441ns-461ns | 1 | 0 | S2 | 0 | 从S4状态输入0，转移到S2 |

**4. 第三组测试序列（1011）：**

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 461ns-481ns | 1 | 1 | S3 | 0 | 检测到101 |
| 481ns-501ns | 1 | 0 | S2 | 0 | 检测到10 |
| 501ns-521ns | 1 | 1 | S3 | 0 | 检测到101 |
| 521ns-541ns | 1 | 1 | S4 | 1 | 检测到1011，输出高电平 |

**5. 第四组测试序列（0000）：**

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 541ns-561ns | 1 | 0 | S2 | 0 | 从S4状态输入0，转移到S2 |
| 561ns-581ns | 1 | 0 | IDLE | 0 | 从S2状态输入0，转移到IDLE |
| 581ns-601ns | 1 | 0 | IDLE | 0 | 保持IDLE状态 |
| 601ns-621ns | 1 | 0 | IDLE | 0 | 保持IDLE状态 |

**6. 第五组测试序列（1011011）：**

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 621ns-641ns | 1 | 1 | S1 | 0 | 检测到第一个1 |
| 641ns-661ns | 1 | 0 | S2 | 0 | 检测到10 |
| 661ns-681ns | 1 | 1 | S3 | 0 | 检测到101 |

| 时间范围 | Rst_n | data_in | 状态 | data_out | 说明 |
|---|---|---|---|---|---|
| 681ns-701ns | 1 | 1 | S4 | 1 | 检测到1011，输出高电平 |
| 701ns-721ns | 1 | 0 | S2 | 0 | 从S4状态输入0，转移到S2 |
| 721ns-741ns | 1 | 1 | S3 | 0 | 检测到101 |
| 741ns-761ns | 1 | 1 | S4 | 1 | 检测到1011，输出高电平 |

# 五、实验思考

## 1. 遇到的问题及解决方法

1. 问题描述：若输出采用时序逻辑寄存，是否会导致输出相对于状态延迟一个时钟周期？
   解决方法：采用组合逻辑实现输出，使其无需等待一个时钟周期延迟。

2. 问题描述：如何设计使得状态机能够处理重叠序列检测？ 解决方法：通过最长后缀匹配原则设计状态跳转，使检测到1011后根据输入跳转到S1或S2，从而支持重叠检测。

## 2. 实验心得

- 掌握了有限状态机的设计与实现方法，包括状态转移图的绘制、状态编码的选择、状态转移逻辑的设计以及输出逻辑的实现。
- 了解了One-hot编码的优势，以及如何通过根据输入跳转处理重叠序列的情况。
- 学习了使用Vivado进行仿真验证，观察波形结果并分析状态机的行为。

# 六、实验评价

## 1. 自我评价

- 实验完成度：□***优秀*** □良好 □一般 □待提高
- 掌握程度：□***很好*** □较好 □一般 □需要加强

## 2. 实验反馈

1. 实验内容难度：□偏难 □***适中*** □偏易
2. 实验时间安排：□充足 □***适中*** □紧张

## 3. 实验建议

- `fsm_sim.v`文件中出现了一处注释错误：

```
// 等待20个时钟周期+1ns后释放复位
#(`clk_period*5 + 1);
Rst_n = 1;         // 释放复位
#(`clk_period*10);// 等待20个时钟周期
```

提交时已经将注释中的“20个时钟周期”分别修改为“5个时钟周期”和“10个时钟周期”，以与代码中的实际等待时间一致。